Practice midterm — worked answers

Attempt the exam first. Credit should reflect the stated reasoning, not just matching final numbers. Point allocations below follow the question parts; equivalent correct notation or derivations earn credit.

Q1 — 10 points

a (6): Features are independent **conditional on class**. Priors are 0.4 Yes and 0.6 No. Yes conditionals are 6/8=0.75, 2/8=0.25, 4/8=0.5. No conditionals are 3/12=0.25, 9/12=0.75, 8/12=2/3.

Scores: $s_Y = (0.4)(0.75)(0.25)(0.5) = 0.0375$; $s_N = (0.6)(0.25)(0.75)(2/3) = 0.075$. Predict No under 0/1 loss. Suggested credit: assumption 1, priors/conditionals 3, scores/prediction 2.

Explanation: Bayes' rule says $P(c|x) = P(c)P(x|c)/P(x)$. Conditional independence lets us replace the joint feature likelihood by $P(\text{Red}|c)P(\text{Domestic}|c)P(\text{SUV}|c)$. For example, the denominator of $P(\text{Domestic}|Y) = 2/8$ is the number of Yes examples, not all 20 examples. Independence is assumed within each class; the features need not be independent in the full population. The scores omit the shared denominator $P(x)$, so comparing them gives the same ranking as comparing posteriors. Under 0/1 loss every mistake costs the same, so choose the larger posterior.

b (3): $P(Y|x) = 0.0375/0.1125 = 1/3$, $P(N|x) = 2/3$. Risk of predicting Yes is $1(2/3) = 2/3$; risk of predicting No is $4(1/3) = 4/3$. Predict Yes. Threshold is 1/(1+4)=0.2, so the posterior exceeds it. One point each for posterior, risks, decision.

Explanation: The normalizing constant is $0.0375 + 0.075 = 0.1125$. For each proposed action, average its possible costs over the true class. Predicting Yes costs nothing when Yes is true and 1 when No is true; predicting No costs 4 when Yes is true. If $q = P(Y|x)$, the two risks are $1 - q$ and $4q$. Choose Yes when $1 - q < 4q$, or $q > 0.2$. This explains why the cost-sensitive decision changes even though No remains the more probable class: missing a Yes is much more expensive. At the threshold the actions tie.

c (1): $(0 + 1)/(8 + 2) = 0.1$. The denominator includes the two possible Type values, not the number of features.

Explanation: For a categorical feature with K possible values, add-one smoothing uses $(n_{v,c}+1)/(n_c+K)$. Here $K = 2$ because Type is SUV or Sports. Adding one pseudo-count to each category adds two to the denominator and prevents an unseen category from making the entire class score zero. The smoothed SUV and Sports probabilities would be 1/10 and 9/10, which still sum to one.

Q2 — 14 points

a (6):

$$L(\theta) = (2\theta/3)^3(\theta/3)^3[2(1-\theta)/3]^1[(1-\theta)/3]^1 = \frac{16}{3^8}\theta^6(1-\theta)^2.$$

$$\ell = \log(16/3^8) + 6\log\theta + 2\log(1-\theta), \qquad \ell' = 6/\theta - 2/(1-\theta).$$

Setting the derivative to zero gives $6(1 - \theta) = 2\theta$, so $\hat{\theta} = 3/4$. $\ell'' = -6/\theta^2 - 2/(1-\theta)^2 < 0$ on (0,1), and both endpoints give likelihood zero because both groups occur. Credit: likelihood 2, log/derivative 2, estimate/maximum check 2. A count-summary likelihood can contain an additional theta-independent multinomial coefficient without changing the estimate.

Explanation: Independent observations contribute multiplicatively to the likelihood; repeated observations therefore become exponents. The three zeros and three ones supply six factors of θ, while the observations of 2 and 3 supply two factors of $1-\theta$. The factors of 2 occur four times, giving $2^4/3^8$. Taking

logs turns the product into a sum without changing its maximizer because log is increasing. Intuitively, θ is the total probability of the group $\{0, 1\}$, so its estimate is the observed fraction in that group, 6/8. The fixed 2:1 proportions within each group do not affect that estimate.

b (3): $\ell = -N\log(2b) - S/b$ with $S = \sum|x_i|$. $\ell' = -N/b + S/b^2 = 0$ gives $\hat{b} = S/N$. Here S=8, N=4, so b=2. The derivative is $(S - Nb)/b^2$, positive below S/N and negative above, verifying the maximum. Credit: objective/derivative 1, estimator 1, calculation 1.

Explanation: Start with $L(b) = (2b)^{-N}\exp(-\sum_i |x_i|/b)$. Taking its log gives the stated objective. Setting the derivative to zero and multiplying by $b^2 > 0$ gives $-Nb+S = 0$. For these data $S = |-2|+|0|+|1|+|5| = 8$. Thus the Laplace scale is the mean absolute distance from its fixed center zero. Do not substitute the sample mean as the center: this problem only estimates b.

c (3): With v=sigma^2, $\ell = -N\log(2\pi v)/2 - \sum(x_i - \mu)^2/(2v)$. The derivatives are $\sum(x_i - \mu)/v$ and $-N/(2v) + \sum(x_i - \mu)^2/(2v^2)$. Thus mu=2, v=(1+0+1)/3=2/3. The likelihood optimization produces N; N-1 is an unbiased-variance correction rather than this MLE. Sigma would be $\sqrt{2/3} = 0.816$. Credit: derivative reasoning 1, estimates 1, denominator explanation 1.

Explanation: Setting the mean derivative to zero gives $\sum_i x_i - N\mu = 0$, hence $\hat{\mu} = (1 + 2 + 3)/3 = 2$. Setting the variance derivative to zero and multiplying by $2v^2$ gives $Nv = \sum_i(x_i - \mu)^2$. Substitute the estimated mean: squared deviations are $1, 0, 1$, so $\hat{v} = 2/3$. The log likelihood is maximized in the mean by minimizing squared deviations; with that mean fixed, its variance derivative changes from positive to negative at 2/3. Dividing by $N - 1$ instead would estimate variance without bias across repeated samples, a different goal from maximizing this sample's likelihood.

d (2): $\ell = (\sum x_i)\log\lambda - N\lambda - \sum\log(x_i!)$. $\ell' = \sum x_i/\lambda - N = 0$ gives lambda=8/4=2. Second derivative is -8/lambda^2<0. Credit: log/derivative 1, rate 1.

Explanation: Multiplying the Poisson probabilities combines the powers into $\lambda^{\sum_i x_i}$ and the exponentials into $e^{-N\lambda}$. The factorial terms depend only on the observed data, so their log derivatives are zero. Solving $\sum_i x_i/\lambda = N$ gives the sample mean, $\hat{\lambda} = (0+1+3+4)/4 = 2$. A rate of two means the fitted distribution has expected count two per observation.

Q3 — 12 points

a (4): Product Gaussian density is $\prod_i(2\pi\sigma_i^2)^{-1/2}\exp[-(y_i - \mu_i)^2/(2\sigma_i^2)]$. Taking negative log:

$$NLL = \frac{1}{2}\sum_i[\log(2\pi) + s_i + (y_i - \mu_i)^2 e^{-s_i}].$$

The s term penalizes arbitrarily large variance; without it, increasing variance makes residual penalties approach zero. For fixed common variance, the variable objective is a positive constant times sum squared error, equivalent to MSE minimization. Credit: density/log derivation and expression 2, variance term 1, MSE condition 1.

Explanation: For one example, negative log density is $\frac{1}{2}\log(2\pi)+\frac{1}{2}\log\sigma_i^2+(y_i-\mu_i)^2/(2\sigma_i^2)$. Since $\sigma_i^2 = e^{s_i}$, its reciprocal is e^{-s_i}; summing over examples yields the answer. The residual term charges more for errors made with small claimed uncertainty. The log-variance term charges for claiming large uncertainty, balancing that incentive. If all variances equal the same fixed $v > 0$, then $NLL = C + \text{SSE}/(2v) = C + N\text{MSE}/(2v)$, so the mean predictions that minimize it are exactly the MSE minimizers. With varying variances, the residuals instead receive different weights.

b (5): Class 0 positive: TP=30, FP=5, TN=55, FN=10. Accuracy=85/100=0.850. Precision=30/35=0.857; recall=30/40=0.750; F1=60/(60+5+10)=0.800. Class-1 F1=110/(110+10+5)=0.880. Macro-F1=(0.800+0.880)/2=0.840. Suggested credit: counts 1, accuracy 1, precision/recall 1, class-0 F1 1,

macro-F1 1.

Explanation: Read positive as the named class, not automatically the numeric label 1. TP is true 0 predicted 0; FP is true 1 predicted 0; FN is true 0 predicted 1; TN is true 1 predicted 1. Precision asks what fraction of the 35 predicted zeros were correct. Recall asks what fraction of the 40 actual zeros were found. Use $F1 = 2TP/(2TP + FP + FN)$ to avoid rounding precision and recall early. For class 1 as positive, TP becomes 55, FP becomes 10, and FN becomes 5. Macro-F1 gives each class equal weight, irrespective of its support; accuracy instead counts all correct examples.

c (1): ROC needs scores to sweep thresholds; hard labels supply only one operating point. Select the class-0 column according to `model.classes_` (first column when classes are [0,1]), and mark class 0 as positive consistently.

Explanation: For each threshold τ, predict class 0 when its score is at least τ, then compute TPR and FPR with 0 treated as positive. Sweeping τ traces the ROC curve. Hard labels discard the ranking information, leaving only a single nontrivial threshold operating point. The score column must describe the same positive event as the evaluation labels; using the class-1 probability reverses the ranking for this task.

d (2): Errors [1,0,-1], so MAE=2/3=0.667, RMSE=$\sqrt{2/3} = 0.816$. SSE=2, SST=2, R^2=0. Negative R^2 means SSE>SST: worse than predicting the evaluated targets' mean. It is not "negative squared error" or a negative correlation by definition. Credit: metrics 1, explanation 1.

Explanation: Absolute errors are $[1, 0, 1]$ and squared errors are also $[1, 0, 1]$. Average before taking the square root for RMSE. For $R^2 = 1-\text{SSE/SST}$, the target mean is 2 and $\text{SST} = (1-2)^2+(2-2)^2+(3-2)^2 = 2$. The constant prediction 2 exactly matches this baseline, giving $R^2 = 0$. A negative value would mean the model's squared-error sum exceeds this baseline's; it can be arbitrarily negative.

Q4 — 10 points

a (6): Parent entropy H(1/2)=1. A: weighted child entropy $\frac{4}{8}(0.8113)+\frac{4}{8}(0.8113) = 0.8113$; IG(A)=0.1887. B: $\frac{2}{8}(0) + \frac{6}{8}(0.9183) = 0.688725$; IG(B)=0.311275. Choose B. To three decimals: A entropy 0.811, gain 0.189; B entropy 0.689, gain 0.311. Credit: parent 1, A 2, B 2, root 1. Subtracting unweighted child entropies is incorrect.

Explanation: Binary entropy is $H(p) = -p\log_2 p - (1-p)\log_2(1-p)$, with $0\log_2 0 = 0$. The parent is evenly split, so its label uncertainty is one bit. A's two branches have positive fractions 3/4 and 1/4, which have equal entropy by symmetry. B's first branch is pure and contributes zero; its other branch has positive fraction $2/6 = 1/3$. Weight each entropy by the probability of reaching that branch, namely its fraction of the eight training examples. Information gain is parent uncertainty minus this expected remaining uncertainty. B leaves less uncertainty and therefore has greater gain.

b (2): An ID isolates rows into pure singleton leaves and gets IG=1 on this training set, but does not provide reusable discrimination for unseen IDs. Use suitable feature selection, constraints/pruning, and held-out evaluation; gain ratio can reduce many-valued split preference. An unsplittable mixed leaf predicts majority class, with an explicit tie policy if necessary.

Explanation: A separate branch for every ID memorizes each observed label, making training entropy zero. That says little about a new ID with no training branch. Held-out evaluation exposes this failure, while restricting such splits or pruning can curb memorization. At a mixed leaf that cannot split usefully, the majority class minimizes training 0/1 error: predicting the minority would misclassify more examples. If counts tie, use a declared deterministic policy rather than pretending the features separate the labels.

c (2): RF aggregates independently grown randomized/bootstrap trees; a cascade feeds forest class-probability outputs into later layers, typically alongside original features. Out-of-fold training probabilities

are produced by models that did not fit the example they predict, reducing leakage/overconfident self-prediction in the augmented representation. Validation chooses cascade depth. One point each for structure and OOF explanation.

Explanation: In a standard RF, trees vote or average probabilities at the same level; one tree's predictions are not inputs to another tree. In a cascade, an earlier layer's class probabilities become features for a later layer. To construct those features for training rows, partition the training data into folds, fit an earlier-layer model on the other folds, and predict the held-out fold; repeat until every row has a prediction made without fitting that row. In-sample predictions can encode memorization that will be unavailable for new examples, misleading the later layer.

Q5 — 14 points

a (6):

$$Y_{00} = 1(1) + 2(-1) + 0(0) + 1(2) + 1 = 2,$$
$$Y_{01} = 2(1) + 0(-1) + 1(0) + 3(2) + 1 = 9,$$
$$Y_{10} = 0(1) + 1(-1) + 2(0) + 1(2) + 1 = 2,$$
$$Y_{11} = 1(1) + 3(-1) + 1(0) + 0(2) + 1 = -1.$$

Convolution output is $\begin{bmatrix} 2 & 9 \\ 2 & -1 \end{bmatrix}$, ReLU output $\begin{bmatrix} 2 & 9 \\ 2 & 0 \end{bmatrix}$, pooled output [9] of shape 1×1×1. Credit: one per correctly demonstrated convolution entry, one ReLU, one pooling. Bias is added once per output cell.

Explanation: A 2×2 kernel fits at two horizontal and two vertical positions in a 3×3 image, producing a 2×2 map. At each position, multiply matching patch and kernel entries, sum all four products, and add the single bias. For example, the top-right patch is $\begin{bmatrix} 2 & 0 \\ 1 & 3 \end{bmatrix}$, yielding $2+0+0+6+1 = 9$. Cross-correlation preserves the displayed kernel orientation. ReLU applies $\max(0, u)$ to each entry independently, replacing only the -1 here. One pooling window then covers the entire resulting map and returns its largest value, 9.

b (6): Conv spatial size $(8-3)/1 + 1 = 6$, output 3×6×6. Pool spatial size $\lfloor (6-2)/2 \rfloor + 1 = 3$, output 3×3×3. Flatten length=27; dense output length=4. Each convolution filter has 2×3×3=18 weights and 1 bias; total 3×19=57. Pool parameters=0. Dense parameters=27×4+4=112. Network total=169. Suggested credit: conv shape 1, pool/flatten/output shapes 2, conv per-filter/total 1, pool 0 and dense 1, total 1.

Explanation: Use channel-first shapes (C, H, W), omitting the batch dimension. With dilation one, each spatial output dimension is $\lfloor (n + 2p - k)/s \rfloor + 1$. Each of the three filters sees both input channels and creates one 6×6 output map. Pooling works separately in each channel, so it reduces height and width without changing the three channels. Flattening merely rearranges the $3 \cdot 3 \cdot 3 = 27$ activations. Parameter counts depend on learned coefficients, not the number of activation locations: the same 19 parameters per convolution filter are reused at every position. Each of four dense outputs needs 27 independent weights and one bias, giving $4(27 + 1) = 112$. Pooling and flattening learn nothing.

c (2): Local receptive fields restrict each output to a patch; shared filter coefficients are reused at spatial positions instead of independently learning every spatial connection. Under groups=1, each output filter spans all input channels; channel contributions are summed. One point each.

Explanation: Local connectivity means a filter examines a small spatial neighborhood instead of the entire image. Weight sharing means the same detector is applied at every valid spatial position; it does not learn a new set of weights for each patch. For two input channels and a 3×3 kernel, an ordinary filter has shape $2 \times 3 \times 3$. Sum its weighted contributions from both channels before adding its bias to obtain one output value. Multiple filters learn different detectors and create different output channels.

Q6 — 16 points

a (6):

$$\frac{\partial L}{\partial p} = -\frac{y}{p} + \frac{1-y}{1-p}, \quad \frac{\partial p}{\partial z} = p(1-p), \quad \delta = p - y.$$

$$g_3 = \delta h_1, \quad g_4 = \delta h_2, \quad g_1 = \delta w_3 h_1(1-h_1)x, \quad g_2 = \delta w_4 h_2(1-h_2)x.$$

Update all $w_i \leftarrow w_i - \eta g_i$ using pre-update weights. For half squared error, delta becomes $(p-y)p(1-p)$, while downstream chain-rule factors keep the same structure. Credit: BCE cancellation 1, output gradients 1, hidden gradients 2, update 1, squared-error delta 1.

Explanation: First simplify the output error by the chain rule: $\partial L/\partial z = [-y/p + (1-y)/(1-p)]p(1-p) = -y(1-p) + (1-y)p = p - y$. For w_1, trace the path $w_1 \to h_1 \to z \to p \to L$: its factors are input x, hidden sigmoid derivative $h_1(1-h_1)$, output weight w_3, and output delta δ. The w_2 path uses h_2, w_4 instead. Output weights have no hidden-activation derivative on their own paths: $\partial z/\partial w_3 = h_1$. With half squared error, $\partial L/\partial p = p - y$, so the output sigmoid factor remains rather than cancelling. Compute every gradient at one parameter snapshot before applying gradient descent.

b (4): h1=h2=0.5, z=1(0.5)-1(0.5)=0, p=0.5, delta=-0.5. In weight order (1,2,3,4), gradients are (-0.125,+0.125,-0.25,-0.25). Updated weights are (0.0125,-0.0125,1.025,-0.975). Hidden gradients use the original +1 and -1 output weights. Credit: forward 1, hidden gradients 1, output gradients 1, simultaneous updates 1.

Explanation: Both hidden preactivations are zero, and sigmoid of zero is 1/2. Their contributions to z cancel because the output weights have opposite signs. Thus $p - y = 0.5 - 1 = -0.5$. The hidden sigmoid derivatives are each $0.5(1-0.5) = 0.25$, giving $g_1 = (-0.5)(1)(0.25)(1) = -0.125$ and $g_2 = (-0.5)(-1)(0.25)(1) = 0.125$. The output gradients are each $(-0.5)(0.5) = -0.25$. For example, $w_4^{\text{new}} = -1 - 0.1(-0.25) = -0.975$. A negative gradient makes a weight increase under gradient descent; it does not mean subtracting the gradient makes it more negative.

c (2): $\phi_k'(u) = e^{2ku}/(1+e^{2ku}) = \sigma(2ku)$. Replace hidden factors $h_j(1-h_j)$ by $\sigma(2kw_jx)$; output gradients still use the new hidden activations. As k increases, activation approaches max(0,u). ReLU is differentiable away from zero and uses a selected subgradient at zero, commonly 0. Scaled Softplus is not required to train ReLU networks.

Explanation: Differentiate the logarithm using the chain rule: $(1/(2k))(1/(1+e^{2ku}))(2ke^{2ku})$; the $2k$ factors cancel. For a hidden unit, evaluate this derivative at its preactivation $u = w_jx$, while also recomputing h_j and p with the new activation. For positive u, $\log(1+e^{2ku})$ approaches $2ku$ as positive k grows; for negative u it approaches zero; at zero the activation is $\log 2/(2k) \to 0$. Thus the limiting activation is ReLU. Its derivative is 1 for positive input and 0 for negative input. At zero there is no ordinary derivative, but backpropagation can use a chosen subgradient.

d (3): Chain rule gives $\partial h_L/\partial x = \prod_{l=1}^{L}[w_l\sigma'(w_lh_{l-1})]$, hence magnitude $\leq (1/4)^L$ under the weight bound. Residual block Jacobian is $I + J_F$. Without weight bounds, weights/matrix Jacobians can amplify or destabilize gradients. Residual identity paths help, but learned Jacobians may still cancel or amplify directions, and optimization/generalization problems remain. One point each for bound, residual Jacobian, qualifications.

Explanation: For sigmoid, $\sigma'(u) = \sigma(u)(1-\sigma(u)) \leq 1/4$. Each layer therefore contributes a factor with magnitude at most $|w_l|/4 \leq 1/4$. Multiplying L such factors yields 4^{-L}; already at ten layers this is about 9.54×10^{-7}. This particular bound relies on the stated scalar weight bound: large weights or matrix effects change the analysis. Differentiating the residual sum gives an identity term plus the learned branch Jacobian, so a gradient has a direct identity path. That can improve propagation, but J_F can counteract or amplify the identity; for example, $J_F = -I$ makes the local total Jacobian zero.

e (1): Subtracting almost equal floating-point losses incurs cancellation; division by tiny epsilon magnifies error. Reducing epsilon trades smaller truncation error against larger roundoff effects.

Explanation: The central difference is $[L(w+\epsilon) - L(w-\epsilon)]/(2\epsilon)$. In exact arithmetic its truncation error typically decreases as ϵ^2 for a sufficiently smooth loss. In floating-point arithmetic the two loss values become nearly equal as epsilon shrinks, so their difference can lose significant digits, and dividing by epsilon magnifies the error. Balance the two effects by checking a range of step sizes rather than assuming the smallest is best.

Q7 — 10 points

a (6): Three central defects, two points each for defect plus explanation/correction:

1. Fitting on stacked train/test learns test statistics. Split first, fit only on training features, then transform both with the same fitted scaler.
2. `train_x` and `test_x` retain the final label column. Predictions use test labels in distances: direct target leakage. Separate last-column y and all preceding feature columns before fitting/transforming.
3. `sum(train_x-test_x[0], axis=1)` is a signed feature-difference sum for each training row. It can be negative and cancels opposing differences. Use sum of squared feature differences, optionally square-rooted, over the feature axis.

The code sorts these incorrect scores ascending, retrieves four training rows, and votes using their labels. It predicts only the first test row; that would also be incomplete if used to claim performance on the whole test set.

Explanation: The scaler is meant to put features on comparable scales so large numerical units do not dominate nearest-neighbor distances. Fitting it on the test set lets evaluation data influence those scales, even without using test labels. Including the label column is a second, more direct leak: the unknown answer itself becomes a distance feature. Finally, signed differences are not a distance: a difference vector $[10, -10]$ sums to zero despite being far from zero. Squared Euclidean distance sums nonnegative squared differences, preventing cancellation; its square root is unnecessary for neighbor ranking because square root is increasing. After correcting these issues, sort valid distances, take the four closest rows, and vote using their separate training labels.

b (1): `bincount` counts occurrences by nonnegative integer class value; `argmax` returns the first maximal index, so a 0/1 vote tie favors 0. HW1 requires the class of the nearest neighbor among tied classes. Valid categorical integer labels are assumed here.

Explanation: For neighbor labels $[1, 0, 1, 0]$, the counts for classes 0 and 1 are both two. The first maximal count has index 0, so this code predicts 0 regardless of which neighbor is closest. If the distance-sorted neighbors begin with class 1, the stated HW1 policy instead predicts 1. For more than two classes, restrict the nearest-neighbor tie-break to the classes tied for the largest vote.

c (3): Test-based selection chooses favorable test noise and makes test performance optimistic; repetition preserves the defect. Use test holdout, training/validation separation, train-only preprocessing, validation tuning/early stopping, restore chosen checkpoint, freeze choices, then test. For unseen materials, keep all rows with a material ID in one partition, using identifying metadata; group-aware validation should follow the same objective. One point per bias, workflow, grouped split.

Explanation: Trying many models and taking the best test score uses the test set as a tuning set: a lucky error fluctuation can decide the winner. Repeating this selection five times does not make those chosen test scores independent assessments. Reserve a test partition first, fit preprocessing and model parameters on training data, and use validation data to choose architecture and the stopping checkpoint. Restore that checkpoint and freeze the procedure before the final test evaluation. If deployment targets unseen

materials, split by material identity before preprocessing so related measurements cannot occur on both sides of a split. Otherwise the score may measure recognition of known materials rather than transfer to new ones.

Q8 — 8 points

a (4): Line 1 samples same-shaped standard Gaussian noise. Line 2 directly samples a noisy image at t using fixed cumulative schedule coefficients; it is forward corruption. Line 3 predicts noise from xt and t, learning model parameters. Line 4 averages squared predicted-minus-sampled noise across tensor elements. Target is the sampled epsilon, not x0 or a class label. The fixed schedule and sampled inputs/noise are not model parameters. Timesteps identify corruption strength. One point per line including these roles.

Explanation: The noise tensor has the same dimensions as the clean batch, with fresh standard-normal values. The equation for x_t mixes clean signal and noise: large $\bar{\alpha}_t$ retains more signal, while small $\bar{\alpha}_t$ produces stronger corruption. It samples the desired timestep directly without simulating all preceding noising steps. The model receives only the noisy input and timestep, and must estimate the particular noise realization used to construct that example. Supplying t identifies how much corruption to expect. Backpropagation updates the model weights through its prediction and MSE loss; it does not optimize the sampled target noise or the fixed schedule in this setup.

b (2): sqrt(a)=0.8 and sqrt(1-a)=0.6. xt=0.8(2)+0.6(-1)=1.0. Loss=$(0.5\text{-}(-1))^2$=2.25. Derivative=2(1.5)=3 for this single element; for M averaged elements, each derivative is divided by M. Clean estimate is $(1 - 0.6(0.5))/0.8 = 0.875$. Its mismatch with x0=2 reflects the incorrect noise prediction. Credit: noising/loss 1, derivative/recovery 1.

Explanation: Use the supplied cumulative coefficient itself: $\sqrt{0.64} = 0.8$ and $\sqrt{0.36} = 0.6$. The prediction error is $0.5 - (-1) = 1.5$, so its square is 2.25 and its derivative with respect to the prediction is $2(1.5) = 3$. To recover the clean estimate, rearrange $x_t = \sqrt{a}\, x_0 + \sqrt{1-a}\, \epsilon$ and replace the unknown noise by its prediction: $\hat{x}_0 = (x_t - \sqrt{1-a}\, \hat{\epsilon})/\sqrt{a}$. Using the true noise -1 instead would return $(1 + 0.6)/0.8 = 2$ exactly. This recovery is a clean-image estimate, not by itself the full reverse sampling procedure.

c (2): Training uses clean data, sampled t/noise, and a supervised noise target; generation starts with Gaussian noise and repeatedly applies learned reverse-denoising steps without a known clean target. A vanilla GAN maps latent noise through a generator in a forward pass and trains adversarially against a discriminator; diffusion learns denoising using a fixed corruption process and usually generates through multiple reverse steps. One point each.

Explanation: During training, clean examples let us create noisy inputs with known targets, so this is supervised noise prediction even though the data need no human class labels. During generation, the clean image and target noise are unknown. A sampler starts at high noise and uses timestep-conditioned predictions to progressively reduce it. A vanilla GAN instead trains a generator to fool a discriminator that separates generated from real data; its generator usually maps a latent draw to an image in one forward pass. Both can synthesize new examples, but their training signals and sampling procedures differ.

Q9 — 6 points

a (2): RNN state carries information from earlier inputs; the same recurrent/input weights are reused across timesteps. LSTM forget gate controls old cell retention, input gate controls candidate writing, and output gate controls exposed cell information: $c_t = f_t c_{t-1} + i_t \tilde{c}_t$, $h_t = o_t \tanh c_t$ (elementwise products for vectors). Additive state evolution helps preserve dependencies. One point each for RNN and LSTM.

Explanation: A basic RNN computes $h_t = \phi(W_x x_t + W_h h_{t-1} + b)$, so its hidden state summarizes the preceding sequence rather than storing an exact copy of every input. The same matrices are used at

every timestep, allowing a shared transition rule for different sequence lengths. In an LSTM, gates are elementwise values between zero and one: a forget gate near one retains an old cell component, an input gate near zero prevents new writing, and an output gate near zero hides that component from the exposed hidden state. The additive cell path supports retention when gates permit it, but does not guarantee perfect memory.

b (2): Shuffling preserves nucleotide counts, so the frequency predictor is unchanged. CNN accuracy loss shows sensitivity to order-dependent structure under this perturbation, but does not prove biological motif learning, the necessity of order for any classifier, or generalization to unseen genomic families; distribution shift is an alternative explanation. Retraining on shuffled training data with shuffled validation evaluates what the architecture can learn without original order, rather than only the frozen model's robustness. Credit: supported observation 1, limitation/retraining interpretation 1.

Explanation: A frequency representation depends only on counts divided by sequence length; rearranging positions changes neither, so the logistic model receives identical features. The frozen CNN receives different local windows and ordering, so its changed accuracy demonstrates sensitivity to that perturbation. Accuracy loss alone cannot identify which patterns mattered or whether they are biological rather than dataset artifacts. Retraining and validating on shuffled sequences tests the predictive information learnable after order is removed, while also allowing the model to adapt to the changed input distribution. Keep evaluation partitions controlled so the comparison isolates the role of order.

c (2): Mode collapse generates too few kinds of samples despite individual plausibility. Non-saturating generator loss is $-\mathbb{E}_z \log D(G(z))$, encouraging fake samples to be scored real and avoiding the original saturating objective's weak-gradient issue when D rejects fakes. Denoising AE targets clean x from corrupted input; this diffusion parameterization targets sampled epsilon at t. One point for GAN explanation, one for target distinction.

Explanation: For example, a generator trained on many digit classes might repeatedly produce a few plausible digit types: realistic individual samples can coexist with poor coverage. The non-saturating objective minimizes $-\log D(G(z))$, pushing the discriminator's predicted real probability upward. When that probability is near zero, this objective avoids the weak generator signal of the original saturating formulation; it does not itself guarantee that collapse disappears. A denoising autoencoder directly predicts the clean example from a corrupted one. Here the diffusion network predicts the sampled corruption noise, and the timestep specifies its scale; a clean estimate can then be obtained algebraically as in Q8b.

Diagnose your gaps

Treat the score as a practice diagnostic, not a prediction of your course grade. Missing factors in Q6: redo HW3 Q2 symbolically before substituting numbers. Wrong counts/positive class in Q1/Q3: redraw labeled tables first. Wrong Q5 shapes/parameters: keep an activation-shape column separate from a parameter-count column. Weak Q7/Q9 answers: practice explaining what an experiment can establish, the control, and one alternative explanation. Weak Q8: name the exact model target and distinguish noising, training, and sampling.